RTC Relay Server Infrastructure Study

Paper #51, 6 pages

ABSTRACT

Real-Time Communication (RTC) relay servers are critical for ensuring connectivity when peer-to-peer (P2P) links fail, directly impacting end-to-end latency, reliability, and user quality of experience (QoE). However, commercial RTC platforms keep their relay infrastructure—including deployment locations, targeting logic, and fallback mechanisms—opaque, leaving a gap in our understanding of real-world RTC system design. In this paper, we plot a global relay map for three major RTC applications (Zoom Free/Business, WhatsApp, and Discord) by automating cross-region calls over 13 Azure cloud regions worldwide. From our observations, a denser deployment of relay nodes and a better targeting strategy give WhatsApp on average 69 ms, 31 ms, and 35 ms lower end-to-end latency per call compared with Zoom Free, Zoom Business, and Discord, respectively. These insights provide valuable input for next-generation relay infrastructure design.

1 INTRODUCTION

Real-time communication (RTC) is a cornerstone technology that connects people through interactive audio, video, and screen-sharing, and it underpins everyday services such as Zoom, WhatsApp, and Discord [2, 8, 11, 20]. RTC calls require establishing a stable, low-latency data transmission channel between a caller and a callee. When a direct peer-to-peer (P2P [16, 27]) path cannot be established—e.g., due to NATs [21, 24] or firewalls [4, 25]—the call falls back to relayed transmission via a TURN server [23] (a.k.a., relay server), i.e., every media packet is first sent to a relay which then forwards it to the other endpoint. Relay-mode transmission therefore fundamentally changes the data path and the performance semantics of a call [19, 22, 26].

In this paper, we study how the relay server infrastructure affects RTC call performance and reliability from the perspective of a service provider. A fundamental design decision is the construction of the **relay map**. Given a fixed budget—i.e., a fixed number of servers that can be deployed or rented—providers must decide how to place relay servers globally. One end of the design space favors a large number of geographically distributed points of presence (PoPs), which increases the likelihood that a call can be served by a nearby relay and thus reduces latency, but at the cost of higher operation and maintenance overhead and more complex load balancing. The opposite end favors a small number of centralized locations, which simplifies maintenance and enables coarse-grained load pooling, but often incurs higher latency for a large fraction of users. Beyond relay placement, providers must determine the **relay targeting** strategy—how a specific relay is selected for a given call when multiple candidates are available. Finally, robust **relay fallback** mechanisms are critical for maintaining call reliability when relays or transport paths fail, such as due to server outages or regional network disruptions. Together, relay mapping, targeting, and fallback decisions directly shape call latency, loss characteristics, and end-to-end reliability.

Measuring relay infrastructures and their effect on calls poses two practical challenges. First, we must exercise calls from geographically diverse origins to observe the relay locations globally. Second, we must generate a sufficiently large number of controlled calls so that targeting and fallback behaviors become visible and statistically meaningful. Addressing both requirements simultaneously while using real mobile clients is non-trivial.

To our knowledge, there is no prior cross-platform, global-scale measurement that maps and compares relay infrastructures of popular RTC applications; existing work focuses on protocol compliance, P2P performance, or measurements of individual applications [7, 11, 15, 20]. In this paper, **we present the first measurement and comparison study of commercial RTC relay infrastructures across major applications**. Specifically, we study three widely used RTC platforms—Zoom, Discord, and WhatsApp—further distinguishing between Zoom's Free and Business account tiers. To overcome the above challenges, we (1) connect physical mobile clients to WireGuard VPN endpoints in a set of global cloud regions, (2) automate large numbers of one-on-one (1-on-1) calls using Xcode-based device control to sweep caller–callee region pairs, and (3) capture packet traces at the cloud proxies to identify relay IPs and infer relay selection and fallback behavior.

Summary of findings. Using this framework, we uncover three patterns across applications. **(i) Relay map density varies significantly** (Figure 1): Zoom Free uses only two relay locations, while Zoom Business and Discord are globally distributed and WhatsApp has the densest footprint. **(ii) Relay targeting policies differ** in how caller and callee locations influence selection: WhatsApp uses latency-aware multi-probing to pick the nearest responsive relay, whereas Zoom and Discord assign relays based on caller location. **(iii) Fallback robustness varies widely**: WhatsApp opens 3 UDP and 3 TCP fallback sessions toward 3 different relays, while Discord has no fallback beyond a single UDP session.

(a) Zoom Free **(b) Zoom Business** **(c) Discord** **(d) WhatsApp**

Figure 1: Relay maps for three representative RTC applications. Each red dot denotes a geographic location where at least one relay server IP was detected.

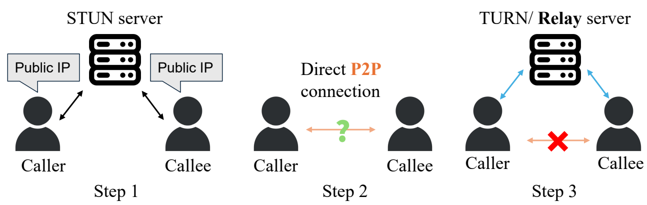

Figure 2: Connection establishment in a 1-on-1 call.

Measured impact on call latency. A denser relay deployment and better targeting shorten call latency. On intra-continent pairs, WhatsApp is on average 144/27/24 ms faster than Zoom Free/Zoom Business/Discord; on inter-continent pairs, WhatsApp is 47/32/39 ms faster.

2 BACKGROUND AND MOTIVATION

RTC applications generally support two call modes: 1-on-1 calls and group calls with multiple participants. In this paper, we focus on 1-on-1 calls for two reasons. First, 1-on-1 calls dominate real-world RTC usage, accounting for over 80% of calls in practice since 2020 [3, 13]. Second, group-call infrastructures are inherently more complex: beyond relay servers, their data transmission paths are also shaped by additional components such as Selective Forwarding Units (SFUs) [29], making it difficult to isolate and quantify the impact of relay infrastructure on call performance through relay-focused measurements alone. Accordingly, we restrict our scope to 1-on-1 calls and defer the study of group-call infrastructures to future work. In this paper, we use the term **relay server** to refer specifically to **TURN servers** that forward media traffic between two client devices.

2.1 How Relay Works in a 1-on-1 Call

Figure 2 illustrates the typical connection establishment of a 1-on-1 call, which sets up a channel for time-sensitive media (e.g., RTP [26]/RTCP [12]) between the two endpoints.

Step 1: Address discovery via STUN. Each client queries a STUN server to learn its *server-reflexive candidate*—the public-facing IP/port assigned by its NAT.

Step 2: Direct P2P connectivity check. The clients run STUN binding checks; if a pair is reachable, Interactive Connectivity Establishment (ICE) [14] selects it and media flows directly over a peer-to-peer (P2P [16]) path with minimal latency.[1]

Step 3: Relay-based fallback using TURN. When direct connectivity fails—e.g., due to symmetric NATs [21, 24], UDP-blocking firewalls [4, 25], or enterprise network policies [10]—an endpoint requests an allocation from a TURN server, which assigns a relayed transport address and forwards media on its behalf. All media packets then traverse the client–relay–client path, so call latency and loss are determined by both legs and the application's relay selection.

2.2 Measurement Framework Overview

Measuring relay infrastructures in commercial RTC applications is challenging for two reasons. First, relay deployment and selection logic are proprietary and closed-source, and applications do not expose internal relay maps or decision processes. Second, relay behavior is highly dynamic: the relay used for a given call may vary with caller and callee locations, call timing, and account type. We therefore adopt a third-party, black-box measurement approach that infers relay infrastructure properties solely from network traffic generated by controlled end-host call experiments.

Application selection. We study three popular RTC applications: Zoom, Discord, and WhatsApp, focusing on their mobile clients and 1-on-1 calls. These applications represent different segments of the RTC ecosystem and have large, diverse user bases. For Zoom, we explicitly distinguish between *Free* [32] and *Business* [30] account tiers for two reasons. [2] First, Zoom's public documentation indicates that paid accounts can opt in or out of specific data-center regions, while Free accounts are restricted to a provisioned region [33]. Second, although Zoom publishes IP address ranges used by its services [31], these disclosures do not specify which IPs

[1]Some RTC applications (e.g., Discord) do not attempt P2P connectivity and always use relay-based media delivery.

[2]Zoom also has a school/education account. We do not include that due to access difficulty.

correspond to TURN or relay servers, nor their roles in media forwarding, making direct measurement necessary to identify relay infrastructure differences across account tiers.
Call generation and traffic collection. To enable large-scale and geographically diverse measurements, we take two steps: 1) We connect physical iPhones to virtual machines deployed in multiple Azure regions using WireGuard VPN tunnels, causing calls to appear as originating from different geographic locations; and 2) We use Xcode-based automation scripts to repeatedly generate 1-on-1 calls across different caller–callee region pairs. All network traffic is captured at the cloud VMs for subsequent analysis.
Relay inference. We analyze the collected network traces to identify relay servers and characterize their behavior. Relay IPs are extracted by isolating RTP flows associated with relay-mode calls [7]. To infer each relay IP's geographic location and the organization operating it, we query IP intelligence services (e.g., ipinfo) [1]. To validate the accuracy of these inferences, we additionally run traceroute and active latency measurements toward the inferred relay IPs and check for consistency between the measured path/RTT characteristics and the reported locations. Together, these techniques enable us to reconstruct each application's relay map, relay targeting policies, and fallback behavior from end-host observations.

2.3 Related Work

Prior work on RTC relays falls into three lines. (i) Measurement studies of individual applications—Zoom, Skype, Google Meet, Webex—characterize call latency, availability, and media behavior under varying networks [5, 6, 18, 28], but treat relaying only as an implicit fallback and do not reconstruct relay maps, targeting, or fallback strategies. (ii) Aggregate measurements quantify how often RTC systems use TURN in the wild [21], without distinguishing between applications or providers. (iii) Systems work optimizes the TURN relay itself for forwarding efficiency and scalability [17], operating at the server-design level rather than on how commercial RTC services actually deploy, select, and orchestrate their relay fleets.

In contrast to these lines of work, no prior study provides a systematic, cross-application measurement of commercial RTC relay infrastructures, nor compares relay maps, relay targeting policies, and fallback mechanisms across major platforms and quantifies their impact on end-to-end call performance and reliability.

3 MEASUREMENT METHODOLOGY

Our objective is to discover as many relay server locations of RTC applications worldwide as possible. Achieving this requires initiating RTC calls from diverse geographic regions. However, it is most challenging to deploy mobile devices across the globe physically. To overcome this, we leverage Azure VMs distributed worldwide to serve as VPN proxies. Each VM hosts a VPN server, and our test devices connect to it via WireGuard [9], forcing all mobile traffic through the assigned VM. This setup allows us to emulate RTC calls originating from the VM's regions. Meanwhile, we block the P2P connection between the VMs to avoid forming P2P transmission during the calls. Finally, we capture packets at the VM using Wireshark and analyze the RTP packets exchanged with relay servers to identify their IP addresses and geographic locations.

Azure node	City / Proximity	Global Region
East US 2	Boydton, Virginia, USA	America
Central US	Des Moines Iowa, USA	America
West US	San Francisco, California, USA	America
South Central US	San Antonio, Texas, USA	America
Chile Central	San Diego, Chile	South America
UK South	London, United Kingdom	Europe
Poland Central	Warsaw, Poland	Europe
UAE North	Abu Dhabi, UAE	Middle East
Japan East	Tokyo (Saitama), Japan	Asia Pacific
Central India	Pune, India	Asia Pacific
South Africa North	Johannesburg, South Africa	Africa
Australia East	Sydney (New South Wales), Australia	Asia Pacific
Malaysia West	Kuala Lumpur, Malaysia	Asia Pacific

Table 1: Selected Azure VM regions, their proximity cities, and Azure global region categories.

3.1 Call Experiment Setup

3.1.1 Device and Network Setup. We selected two iPhone 11 devices as the caller and the callee. Each phone connects to an Azure VM through a WireGuard VPN tunnel, configured to forward all mobile traffic through the chosen VM. A controller MacBook laptop orchestrates the experiments. It remotely manages the Azure VMs, starting and stopping traffic captures, issuing commands to block specific IPs, and ensuring synchronization between devices.

3.1.2 Caller and Callee Location Setup. To best emulate caller and callee locations worldwide under the time and resource constraints, we select a representative set of Azure VM regions across different continents, as listed in Table 1. We iterate the caller and callee through each VM, respectively, forming 169 different pairs. Each caller–callee pair is tested by initiating the same call 10 times consecutively. This repetition is necessary because applications do not always select the same relay server across different call attempts. Multiple repetitions provide richer observations and improve the reliability of our measurement results.

3.1.3 Experiment Procedure and Trace Capture. Before each experiment, both phones terminate background applications, connect to WiFi, and route all traffic through assigned Azure

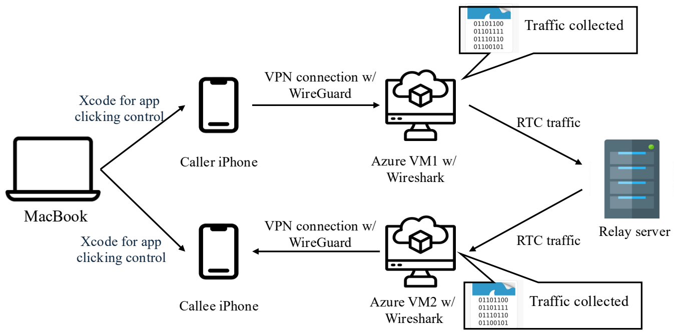

Figure 3: Relay server measurement methodology. Using 2 VMs from Microsoft Azure to mimic the caller and callee location at different places around the globe.

VMs via WireGuard.[3] We implement an automation program on a MacBook using Xcode to control the iPhones, which automatically restarts the RTC application, initiates a call, and maintains the call for one minute with both the microphone and the camera on. The Wireshark trace-capturing commands are also initiated by the MacBook program and sent remotely to the VMs. This process is repeated 10 times for each caller–callee configuration. Each call produces two synchronized packet traces collected at the caller- and callee-side VMs. For each of the three applications studied (with Zoom exercised separately at the Free and Business tiers), this yields roughly 112 hours of packet traces in total.

3.2 Relay Server Measurement

With the data collected above, we can identify the relay servers and analyze their impact on call performance. Relay server identification from packet traces proceeds in two steps. 1) Extract relay server IP addresses from the captured traffic: packet traces collected at the Azure VMs contain a mixture of signaling, background application, and RTC media traffic. To precisely identify relay server IPs, we focus on RTP media packets, which dominate call traffic and, under our P2P-blocked setup, are necessarily exchanged with relay servers. Following the DPI approach in [7], we locate RTP packets using header matching and RTP-specific fields, and extract their destination IPs as relay server IPs. 2) Determine the geographic locations of the identified relays: we query *ipinfo.io* [1] for database-based estimates and further validate each reported location using the multi-vantage-latency and traceroute-based methods detailed in Appendix B. Only locations consistent across these checks are accepted as valid.

To measure the performance impact of the relay server on a call, we use the RTT computed from RTP packets flowing in both directions. This methodology allows us to extract the most information from a call, given the abundant number of RTP packets per call. For every RTP packet detected by the DPI, we extract its send/arrival timestamps at the caller and callee VMs and sum the caller–relay–callee and callee–relay–caller one-way latencies to obtain the packet's end-to-end RTT.[4] The call-level RTT reported in §4.4 is the mean over all RTP packets in the call; each one-minute call yields more than 10,000 RTP packets, making this mean a stable estimate of the call's typical RTT.

4 RELAY ANALYSIS

This section characterizes the relay infrastructures we uncovered using the measurement methodology in §3. We first present the global relay maps observed in each application (§4.1). We then analyze how relays are targeted for each call (§4.2) and how applications fall back to alternative relays or transport sessions upon failures (§4.3). Finally, we quantify how relay placement and relay-selection policies jointly impact end-to-end call latency (§4.4).[5]

4.1 Relay Map

Figure 1 plots all relay servers we detected for Zoom (free and business), Discord, and WhatsApp.[6] Each red dot corresponds to a geographic location (nearest city/country) where at least one relay IP was observed. We refer to relays co-located in the same geographic location (e.g., within one city) as one *relay node*. We acknowledge that the relay maps may be incomplete: our caller/callee vantage points may not fully cover all possible relay deployments. Nonetheless, these maps are produced under the same measurement conditions across the three applications (with Zoom shown at both the Free and Business tiers), and thus offer a consistent view of the *relative* relay geolocation density and deployment footprint.

Zoom provisions two tiers of relay infrastructure, and business accounts are served by substantially more relay nodes. As shown in Figure 1a and Figure 1b, calls between two Zoom Free accounts consistently traverse

[3]We also disable GPS on both phones throughout the experiments: some RTC apps query the device's GPS for geolocation, which would leak the phone's true location and bypass the VPN-based region emulation.

[4]Wireshark stamps each packet from the host VM's system clock, so the two VMs' clocks may be offset by a constant δ. The offset cancels in our estimate: the $+\delta$ bias in the forward one-way delay offsets the $-\delta$ bias in the reverse one-way delay when the two are summed to form RTT.

[5]We focus on latency as our performance metric because it most directly reflects the impact of relay location on a call. Other metrics such as throughput and jitter are confounded by factors orthogonal to relay placement—e.g., video codec adaptation, load balancing, and network conditions at intermediate hops—and therefore provide a less clean signal for comparing relay infrastructures.

[6]For Zoom, the relays shown here include only those used to establish the *initial* media path. We exclude Zoom's fallback relays because they are predominantly hosted on AWS and require additional failure-triggered measurements to reliably enumerate; we analyze Zoom's fallback behavior in §4.3.

only two relay BGP prefixes in the U.S., both announced by Zoom's AS30103: `206.247.0.0/21` (Leesburg/Ashburn, east coast) and `144.195.96.0/21` (San Jose, west coast). In contrast, Zoom Business accounts are served by relays in 12 cities spanning multiple continents: the two U.S. nodes above plus 10 additional cities in Europe, Asia-Pacific, and India. The full list of prefixes and locations is provided in Appendix C. This tiered deployment is consistent with Zoom's documentation [33] and suggests differentiated relay capacity and regional coverage across subscription levels. Most Zoom Business relays are operated on Zoom-owned infrastructure (AS30103); the exception is Zoom's India footprint, where Mumbai (`170.114.104.0/21`) and Hyderabad (`170.114.112.0/21`) are announced by Oracle Cloud (AS31898), indicating that Zoom leases relay capacity from Oracle to serve Indian users.

Discord operates a globally distributed relay fleet built predominantly on third-party CDN and cloud infrastructure. Discord's relays fall into 31 distinct BGP prefixes spanning 15 cities on every inhabited continent (including Johannesburg); the full list of prefixes and locations is provided in Appendix C. The deployment is dominated by Cloudflare PoPs (80% of observations, AS13335), with smaller shares on Google Cloud (16%, AS15169/AS19527) and i3D.net (5%, AS49544, observed only in Tokyo).[7]

WhatsApp exhibits the broadest relay footprint among the studied applications. WhatsApp's relays span 25 cities, with notably higher density in population centers; the full list of IPs, PoP codes, and locations is provided in Appendix C. WhatsApp's relays are consistently hosted on Meta-owned infrastructure, indicating a self-operated global relay backbone. The wide geographic spread suggests WhatsApp aims to keep relays close to end users across most regions, which can reduce relay detours and improve call latency—a hypothesis we quantify in §4.4.

4.2 Targeting Strategy

We define *relay targeting* as the policy by which an application selects the geographic relay node that serves a call, given fixed caller and callee locations. Although the global relay map may contain many relay nodes, only a subset (or one) of these nodes are eligible to serve a particular caller–callee pair. We infer targeting behavior by repeating calls under fixed endpoint locations and observing which relay node is ultimately chosen.

Zoom Business and Discord adopt caller-driven targeting. For both, the selected relay is determined solely by the caller's location.[8] Under a fixed caller region, repeated calls select IPs within the same prefix and city, and blocking that prefix prevents call establishment, confirming no alternative node is considered. For Discord, we additionally observe RTP packets exchanged between the caller and relay *before* the callee answers, indicating that relay selection completes entirely on the caller side—a design that may reduce perceived setup latency.

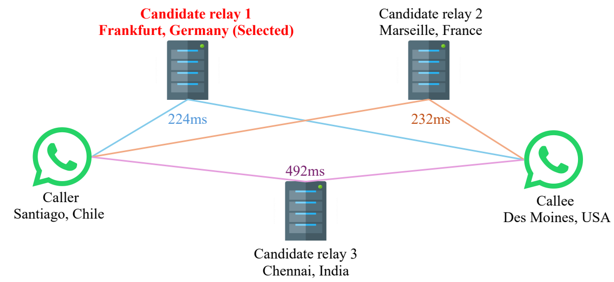

Figure 4: Illustration of WhatsApp's targeting: the relay is selected by minimizing caller–callee RTT. At call start, three candidate relays are offered, and the one minimizing the caller–callee RTT is chosen.

WhatsApp adopts joint caller–callee-aware targeting. As shown in Figure 4, at call setup, both endpoints receive the same set of three TURN server IPs, and each device sends a TURN Allocate request to each candidate and receives the corresponding response. The relay selected for the call is the one minimizing the combined latency observed by the caller and callee, effectively estimating the end-to-end path cost before media transmission. The three candidates and their combined probe RTTs, enumerated across all 169 caller–callee region pairs we tested, are listed in Table 6. Notably, the three TURN server IPs are not fixed for a given caller–callee pair; both the IP addresses and their geographic locations vary across experiments, suggesting an underlying load-balancing mechanism that dynamically adjusts candidate pools while preserving latency-aware selection.

4.3 Fallback Strategy

We define a *fallback session* as a TCP or UDP connection between a client device and a relay IP during a call. RTC systems typically prefer UDP and fall back to TCP under disruption [4, 22]. We count the number of sessions per call (including the default UDP session) to quantify robustness. To trigger fallback, we block relay IP prefixes at the VM during an ongoing call; if the call does not recover within 30 seconds, we consider it terminated. Our measurement results below reveal that Zoom Business and WhatsApp

[7] Our Discord measurements focus on 1-on-1 calls. Discord channel calls (meeting-room style) use separate media servers with a broader deployment footprint; see Discord's published RTC latency dashboard at `https://latency.discord.media/rtc`.

[8] Zoom Free uses only two relay nodes, making targeting analysis less meaningful; we therefore focus on Zoom Business.

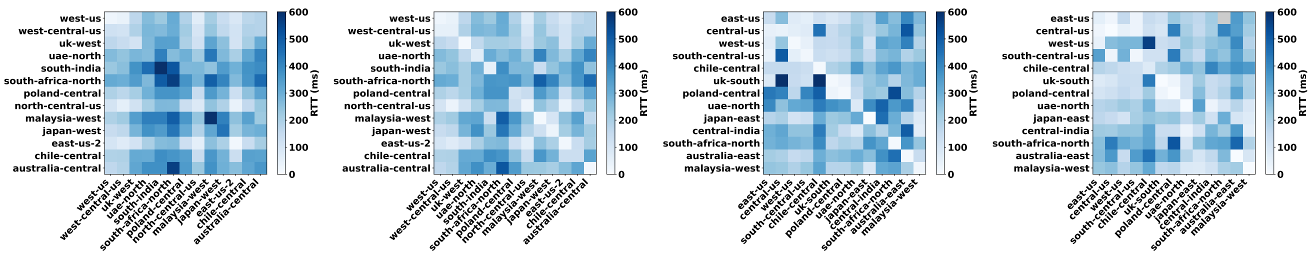

Figure 5: End-to-end call RTT (ms) across 13×13 caller–callee region pairs. Darker cells indicate higher RTT; lighter cells indicate lower RTT. Overall, Zoom Free > Zoom Business ≈ Discord > WhatsApp: Zoom Free is the slowest because of its sparse relay footprint (only two U.S. nodes), while WhatsApp is the fastest because its joint caller–callee relay targeting consistently selects a near-optimal relay for each call.

exhibit substantially richer fallback mechanisms than Zoom Free and Discord.

Zoom Free. One UDP and one TCP session to the same relay IP; after blocking both sessions, the call terminates.

Zoom Business. One UDP and one TCP session to the same relay IP; subsequent blocking triggers additional TCP sessions to AWS-hosted servers that are not used under normal conditions, making fallback effectively unbounded.

Discord. One UDP session only; blocking it immediately terminates the call.

WhatsApp. Three UDP sessions to three candidate relays; upon UDP failure, three corresponding TCP sessions are attempted, yielding up to six sessions before termination.

4.4 Call Latency Analysis

Figure 5 shows the average end-to-end RTT for all 13×13 caller–callee region pairs across the three applications. The heatmaps may be asymmetric across the diagonal due to factors such as caller-driven targeting, load-balancing, and asymmetric underlay routing.

A denser relay deployment primarily benefits geographically close calls. We split the 169 region pairs into *intra-continent* (39 pairs, geographically close) and *inter-continent* (130 pairs, geographically far). For a fair comparison, we fix each caller–callee pair and measure *per-pair* how much faster Zoom Business, Discord, and WhatsApp are than Zoom Free on the same pair, then average those per-pair deltas within each group. On intra-continent pairs, Zoom Business / Discord / WhatsApp are on average **117.2 / 119.7 / 144.0 ms** faster than Zoom Free (**47–58%** per-pair reduction). On inter-continent pairs, however, the savings shrink to **14.7 / 8.0 / 46.7 ms** (**3–17%**): a denser relay map shortens the client–relay leg that dominates short-distance calls, whereas the long-haul underlay sets a hard floor that denser relays cannot overcome for long-distance calls.

A smarter targeting strategy further reduces latency on top of a dense deployment. Zoom Business and Discord have comparably broad global footprints and both use caller-driven targeting; holding the caller–callee pair fixed and averaging the per-pair RTT delta, they differ by only **4.5 ms** on average, despite Zoom Business running on self-owned infrastructure and Discord on Cloudflare. WhatsApp instead adopts joint caller–callee-aware targeting (§4.2), picking the relay minimizing the *combined* caller–callee path cost. For a fair comparison, we again fix each caller–callee pair and measure the per-pair RTT delta: WhatsApp is on average **30.8 ms / 35.4 ms** faster than Zoom Business / Discord per pair. We attribute this additional advantage to WhatsApp balancing both legs rather than conditioning only on the caller's location.

5 CONCLUSION AND DISCUSSION

We measured the relay infrastructures of Zoom, Discord, and WhatsApp, and found that both denser relay deployment and smarter relay targeting are effective levers for reducing end-to-end call latency. The three applications also differ in how relays are hosted—Zoom and WhatsApp operate self-owned relay backbones, while Discord outsources to Cloudflare and other cloud/CDN edges—and in fallback robustness, ranging from WhatsApp's 3-relay UDP+TCP multi-session design to Discord's single-UDP session with no fallback. Our measurements were conducted between August 2025 and March 2026; as providers expand or reconfigure their relay footprints over time, specific numbers reported here (relay counts, city lists, targeting behavior) may shift. Building on this study, we encourage future measurements to further broaden the view by (1) using more diverse calling locations, (2) varying client devices (e.g., smartphones, laptops), and (3) covering additional RTC applications.

REFERENCES

[1] 2025. IPinfo. https://ipinfo.io/. Accessed: 2025-09-25.

[2] Abdullah Azfar, Kim-Kwang Raymond Choo, and Lin Liu. 2016. Android mobile VoIP apps: a survey and examination of their security and privacy. *Electronic Commerce Research* 16, 1 (mar 2016), 73–111. doi:10.1007/s10660-015-9208-1

[3] Anastasia Belyh. 2023. Video conferencing statistics for 2024. https://www.founderjar.com/video-conferencing-statistics/

[4] Sumanth Bhat, Ashwin Rao, and Henning Schulzrinne. 2023. Measuring WebRTC Performance in the Wild. In *Proceedings of the ACM Internet Measurement Conference*.

[5] Xiao Sophia Chang, Bernard Jansen, and Zhi-Li Zhang. 2021. Can You See Me Now? A Measurement Study of Zoom, Webex, and Meet. In *Proceedings of the ACM Internet Measurement Conference (IMC)*.

[6] Kai Chen et al. 2018. A Measurement Study of Skype Call Performance. In *Proceedings of the ACM Internet Measurement Conference (IMC)*.

[7] Peiqing Chen, Peng Qiu, Lambda, and Zaoxing Liu. 2025. Protocol Compliance in Popular RTC Applications. In *Proceedings of the 2025 ACM Internet Measurement Conference*. 294–307.

[8] Discord. 2022. Discord Introduction. https://discord.com/company

[9] Jason A. Donenfeld. 2017. WireGuard: Next Generation Kernel Network Tunnel. In *NDSS Workshop on Binary Analysis Research (BAR)*. https://www.wireguard.com/papers/wireguard.pdf

[10] Jordan Holland, Theophilus Benson, and Zhi-Li Zhang. 2017. An Empirical Study of Enterprise Network Traffic. In *Proceedings of the ACM Internet Measurement Conference (IMC)*.

[11] Christer Holmberg, Stefan Hakansson, and Goran Eriksson. 2015. *Web real-time communication use cases and requirements*. Technical Report.

[12] J. Chesterfield J. Ott. 2010. RTP Control Protocol (RTCP) Extensions for Single-Source Multicast Sessions with Unicast Feedback. RFC 5760. https://datatracker.ietf.org/doc/html/rfc5760

[13] Sagar Joshi. 2024. 50 VoIP Statistics to Reveal the Future of Phone Systems. https://learn.g2.com/voip-statistics

[14] Ari Keränen, Christer Holmberg, and Jonathan Rosenberg. 2018. Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal. RFC 8445. doi:10.17487/RFC8445

[15] A. Keränen, C. Holmberg, and J. Rosenberg. 2020. Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal. RFC 8445. https://datatracker.ietf.org/doc/html/rfc8445

[16] Kenji Leibnitz, Tobias Hoßfeld, Naoki Wakamiya, and Masayuki Murata. 2007. Peer-to-peer vs. client/server: Reliability and efficiency of a content distribution service. In *Managing Traffic Performance in Converged Networks: 20th International Teletraffic Congress, ITC20 2007, Ottawa, Canada, June 17-21, 2007. Proceedings*. Springer, 1161–1172.

[17] László Levai et al. 2023. Supercharge WebRTC: Accelerate TURN Services with eBPF/XDP. In *Proceedings of the ACM SIGCOMM Workshop on eBPF and XDP*.

[18] Kyle MacMillan, Salman A. Baset, and Henning Schulzrinne. 2021. Measuring the Performance and Network Utilization of Popular Video Conferencing Applications. In *Proceedings of the ACM Internet Measurement Conference (IMC)*.

[19] Philip Matthews, Jonathan Rosenberg, Dan Wing, and Rohan Mahy. 2008. Session Traversal Utilities for NAT (STUN). RFC 5389. doi:10.17487/RFC5389

[20] Oliver Michel, Satadal Sengupta, Hyojoon Kim, Ravi Netravali, and Jennifer Rexford. 2022. Enabling passive measurement of zoom performance in production networks. In *Proceedings of the 22nd ACM Internet Measurement Conference* (Nice, France) *(IMC '22)*. Association for Computing Machinery, New York, NY, USA, 244–260. doi:10.1145/3517745.3561414

[21] Azeez Raji, Mark Allman, and Robert Beverly. 2021. Measuring the Deployment and Effectiveness of NAT Traversal Techniques. In *Proceedings of the ACM Internet Measurement Conference (IMC)*.

[22] T. Reddy, A. Johnston, R. Mahy, and M. Petit-Huguenin. 2020. Traversal Using Relays around NAT (TURN): Relay Extensions to STUN. RFC 8656. https://datatracker.ietf.org/doc/html/rfc8656

[23] Tirumaleswar Reddy.K, Alan Johnston, Philip Matthews, and Jonathan Rosenberg. 2020. Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN). RFC 8656. doi:10.17487/RFC8656

[24] Jonathan Rosenberg, Joel Weinberger, Christian Huitema, and Rohan Mahy. 2003. STUN: Simple Traversal of User Datagram Protocol (UDP) Through Network Address Translators (NATs). In *Proceedings of the Internet Society Symposium on Network and Distributed System Security*.

[25] Tommy Sandholm and Aki Virtanen. 2013. An Analysis of Firewall Traversal Techniques for Real-Time Communication. In *Proceedings of the IEEE International Conference on Communications (ICC)*.

[26] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson. 2003. RTP: A Transport Protocol for Real-Time Applications. RFC 3550. https://datatracker.ietf.org/doc/html/rfc3550

[27] Varun Singh and Jörg Ott. 2015. WebRTC: A Platform for Real-Time Communication on the Web. In *Proceedings of the ACM SIGCOMM Computer Communication Review*.

[28] Anirudh Sivaraman et al. 2022. Enabling Passive Measurement of Zoom Performance in Production Networks. In *Proceedings of the ACM Internet Measurement Conference (IMC)*.

[29] Magnus Westerlund and Christer Burman. 2015. *RTCWeb Selective Forwarding Unit (SFU) Architecture*. RFC 7667. IETF.

[30] Zoom. 2026. Plans & Pricing for Zoom Workplace. https://zoom.us/pricing Accessed: 2026-02-09.

[31] Zoom. 2026. Zoom Meetings IP Address Ranges. https://assets.zoom.us/docs/ipranges/ZoomMeetings.txt Accessed: 2026-02-09.

[32] Zoom Support. 2026. Plan Types. https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0060423 Accessed: 2026-02-09.

[33] Zoom Support. 2026. Selecting data center for meetings, webinars, whiteboards, notes and docs. https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0060026 Accessed: 2026-02-09.

A ETHICS

This work does not raise any ethical issues.

B RELAY LOCATION VALIDATION

ipinfo.io returns database-backed IP geolocation estimates that can be stale or inaccurate for newly provisioned or frequently migrated relay IPs. For each relay IP we identify in §3.2, we apply two orthogonal validation checks and accept the reported location only when both agree.

Multi-vantage latency check. We initiate a TCP three-way handshake from every Azure VM in our vantage set (Table 1) to the relay's listening port and record the RTT of the SYN/SYN-ACK exchange as the relay-to-VM latency. The VM with the minimum RTT should be the closest to the relay's true location; its geodesic distance to the reported city must also be consistent with the speed-of-light lower bound implied by that RTT. We flag a relay whose minimum-RTT VM is on a different continent than the reported city, or whose minimum-RTT latency is grossly inconsistent with the distance to the reported city (e.g., sub-50 ms RTT to a VM on a different continent).

Traceroute check. From the minimum-RTT Azure VM, we run a UDP traceroute toward the relay IP and collect the reverse-DNS names of the last several hops before the relay. Router hostnames commonly encode an airport or city code (e.g., `iad`, `fra`, `lax`, `nrt`); we feed the hostname sequence and the *ipinfo.io*-reported city into a large language model and ask whether the final hops converge on the reported city. A relay is flagged when the last identifiable city in the traceroute differs from the reported location.

A relay IP's geolocation is accepted only when both checks agree with *ipinfo.io*; the small number of disagreements we encountered were resolved by manually inspecting the RTT matrix and traceroute output together.

C RELAY IPS, PREFIXES, AND LOCATIONS

Tables 2, 3, 4, and 5 list every relay we observed (§4.1). Zoom (Free and Business) and Discord entries are grouped by the BGP prefix in which the relay IPs were announced (from Team Cymru's IP-to-ASN service); WhatsApp entries are shown at per-IP granularity since all its IPs are in Meta's AS32934 and most reverse-resolve to individual `edgeray-*.facebook.com` PoPs. For Discord, when a (prefix, city) pair was seen more than once we still list it once per distinct city; the number of distinct IPs per prefix is given in the underlying CSV in the supplementary materials. Rows are sorted by country, then by city.

BGP prefix	ASN	City	Country
206.247.0.0/21	30103	Leesburg/Ashburn	U.S.
144.195.96.0/21	30103	San Jose	U.S.

Table 2: Zoom Free relay BGP prefixes observed across 169 test calls. All traffic is served by these two U.S. prefixes (Zoom's AS30103).

Zoom Free relays (Table 2).

Zoom Business relays (Table 3).

WhatsApp relays (Table 4).

Discord relays (Table 5).

BGP prefix	ASN	City	Country
159.124.68.0/23	30103	Melbourne	Australia
159.124.100.0/23	30103	Sydney	Australia
159.124.102.0/23	30103	Sydney	Australia
159.124.34.0/23	30103	Frankfurt	Germany
159.124.38.0/23	30103	Frankfurt	Germany
159.124.42.0/23	30103	Frankfurt	Germany
159.124.44.0/23	30103	Frankfurt	Germany
159.124.46.0/23	30103	Frankfurt	Germany
170.114.112.0/21	31898	Hyderabad	India
170.114.104.0/21	31898	Mumbai	India
147.124.114.0/23	30103	Osaka	Japan
170.114.188.0/23	30103	Tokyo	Japan
170.114.190.0/23	30103	Tokyo	Japan
170.114.192.0/23	30103	Tokyo	Japan
170.114.194.0/23	30103	Tokyo	Japan
170.114.196.0/23	30103	Tokyo	Japan
170.114.198.0/23	30103	Tokyo	Japan
170.114.200.0/23	30103	Tokyo	Japan
159.124.12.0/23	30103	Amsterdam	Netherlands
159.124.14.0/23	30103	Amsterdam	Netherlands
159.124.2.0/23	30103	Amsterdam	Netherlands
159.124.4.0/23	30103	Amsterdam	Netherlands
159.124.6.0/23	30103	Amsterdam	Netherlands
159.124.8.0/23	30103	Amsterdam	Netherlands
170.114.160.0/23	30103	Singapore	Singapore

BGP prefix	ASN	City	Country
170.114.162.0/23	30103	Singapore	Singapore
170.114.164.0/23	30103	Singapore	Singapore
170.114.166.0/23	30103	Singapore	Singapore
170.114.168.0/23	30103	Singapore	Singapore
170.114.170.0/23	30103	Singapore	Singapore
170.114.174.0/23	30103	Singapore	Singapore
170.114.180.0/23	30103	Singapore	Singapore
170.114.182.0/23	30103	Singapore	Singapore
206.247.16.0/21	30103	Centreville	U.S.
206.247.0.0/21	30103	Leesburg	U.S.
206.247.128.0/21	30103	Leesburg	U.S.
206.247.16.0/21	30103	Leesburg	U.S.
206.247.24.0/21	30103	Leesburg	U.S.
206.247.32.0/21	30103	Leesburg	U.S.
206.247.40.0/21	30103	Leesburg	U.S.
206.247.56.0/21	30103	Leesburg	U.S.
206.247.64.0/21	30103	Leesburg	U.S.
206.247.8.0/21	30103	Leesburg	U.S.
144.195.0.0/21	30103	San Jose	U.S.
144.195.128.0/21	30103	San Jose	U.S.
144.195.16.0/21	30103	San Jose	U.S.
144.195.24.0/21	30103	San Jose	U.S.
144.195.32.0/21	30103	San Jose	U.S.
144.195.8.0/21	30103	San Jose	U.S.

Table 3: Zoom Business relay BGP prefixes observed across our test calls. 279 unique IPs spanning 49 (prefix, city) entries in 12 cities. Most are in Zoom's own AS30103; two Indian-city prefixes (Mumbai, Hyderabad) are announced by Oracle Cloud (AS31898), which hosts Zoom's Indian relays.

Relay IP	City	Country
57.144.207.54	Buenos Aires	Argentina
157.240.8.51	Sydney	Australia
31.13.91.62	Rio de Janeiro	Brazil
57.144.247.54	Santiago	Chile
163.70.152.62	Bogotá	Colombia
157.240.196.62	Marseille	France
57.144.121.54	Marseille	France
157.240.0.62	Frankfurt	Germany
157.240.253.62	Frankfurt	Germany
57.144.245.54	Frankfurt	Germany
57.144.249.54	Frankfurt	Germany
57.144.55.54	Chennai	India
163.70.143.62	Mumbai	India
163.70.144.62	Mumbai	India
57.144.125.54	Mumbai	India
57.144.243.54	Mumbai	India
57.144.43.54	Mumbai	India
157.240.203.62	Milan	Italy
31.13.86.48	Milan	Italy
157.240.31.62	Tokyo	Japan
31.13.82.48	Tokyo	Japan
57.144.45.54	Tokyo	Japan
57.144.223.54	Amsterdam	Netherlands
157.240.13.51	Singapore	Singapore
157.240.15.62	Singapore	Singapore
57.144.145.54	Singapore	Singapore
57.144.15.54	Singapore	Singapore
57.144.151.54	Singapore	Singapore
57.144.153.54	Singapore	Singapore
57.144.161.54	Singapore	Singapore
57.144.187.54	Singapore	Singapore
157.240.215.62	Seoul	South Korea
31.13.76.62	Seoul	South Korea
57.144.189.54	Seoul	South Korea
157.240.17.62	Zürich	Switzerland
157.240.214.62	London	U.K.
163.70.151.62	London	U.K.
57.144.239.54	London	U.K.
57.144.241.54	London	U.K.
31.13.66.53	Ashburn	U.S.
57.144.75.54	Ashburn	U.S.
57.144.133.54	Atlanta	U.S.
57.144.253.54	Atlanta	U.S.
57.144.79.54	Atlanta	U.S.
157.240.254.62	Chicago	U.S.
57.144.175.54	Chicago	U.S.
57.144.205.54	Chicago	U.S.
31.13.93.52	Dallas	U.S.
57.144.195.54	Dallas	U.S.
57.144.201.54	Dallas	U.S.
157.240.11.51	Los Angeles	U.S.
57.144.163.54	Miami	U.S.
57.144.135.54	Phoenix	U.S.
157.240.22.52	San Jose	U.S.
57.144.221.54	San Jose	U.S.
57.144.217.54	Seattle	U.S.

Table 4: All WhatsApp relay IPs observed across 169 test calls, with nearest city and country. 56 unique IPs across 25 cities; none are in Africa. All IPs are in Meta's AS32934.

BGP prefix	ASN	City	Country
104.29.134.0/24	13335	Sydney	AU
104.29.139.0/24	13335	Sydney	AU
104.29.143.0/24	13335	Sydney	AU
35.213.192.0/18	19527	Sydney	AU
104.29.143.0/24	13335	Santiago	CL
104.29.146.0/24	13335	Santiago	CL
104.29.157.0/24	13335	Santiago	CL
34.0.48.0/20	15169	Santiago	CL
104.29.144.0/24	13335	London	GB
104.29.151.0/24	13335	London	GB
104.29.156.0/24	13335	London	GB
104.29.159.0/24	13335	London	GB
104.29.132.0/24	13335	Mumbai	IN
104.29.139.0/24	13335	Mumbai	IN
35.207.192.0/18	19527	Mumbai	IN
104.29.132.0/24	13335	Tokyo	JP
104.29.140.0/24	13335	Tokyo	JP
104.29.141.0/24	13335	Tokyo	JP
66.22.208.0/22	49544	Tokyo	JP
66.22.248.0/24	49544	Tokyo	JP
104.29.159.0/24	13335	Warsaw	PL
34.0.240.0/20	15169	Warsaw	PL
104.29.140.0/24	13335	Singapore	SG
104.29.141.0/24	13335	Singapore	SG
104.29.142.0/24	13335	Singapore	SG
35.213.128.0/18	19527	Singapore	SG
104.29.152.0/24	13335	Ashburn	US
104.29.155.0/24	13335	Ashburn	US
104.29.159.0/24	13335	Ashburn	US
104.29.137.0/24	13335	Atlanta	US
104.29.152.0/24	13335	Chicago	US
104.29.153.0/24	13335	Chicago	US
104.29.158.0/24	13335	Chicago	US
104.29.135.0/24	13335	Dallas	US
104.29.137.0/24	13335	Dallas	US
104.29.152.0/24	13335	Dallas	US
104.29.155.0/24	13335	Dallas	US
104.29.158.0/24	13335	Dallas	US
104.29.159.0/24	13335	Dallas	US
34.0.128.0/19	15169	Dallas	US
104.29.154.0/24	13335	Leesburg	US
104.29.155.0/24	13335	Leesburg	US
104.29.157.0/24	13335	Leesburg	US
104.29.159.0/24	13335	Leesburg	US
104.29.135.0/24	13335	San Francisco	US
104.29.135.0/24	13335	San Jose	US
104.29.137.0/24	13335	San Jose	US
104.29.138.0/24	13335	San Jose	US
104.29.143.0/24	13335	San Jose	US
104.29.145.0/24	13335	Johannesburg	ZA
34.1.208.0/20	19527	Johannesburg	ZA

Table 5: Discord relay BGP prefixes observed across our 1-on-1 call measurements, grouped by nearest city. 51 (prefix, city) pairs covering 161 unique IPs across 15 cities, including Johannesburg. AS13335 is Cloudflare; AS15169/AS19527 are Google Cloud; AS49544 is i3D.net.

Table 6: WhatsApp TURN Allocate candidates probed at call setup for each of the 169 caller–callee region pairs. For each call we report the three candidate relays that both endpoints probed. The selected candidate (minimum combined probe RTT, matching the relay WhatsApp ends up using) is in **bold**. Each entry is *city* (*ms*) where ms is host-side Allocate request/response RTT + client-side Allocate request/response RTT, approximating the caller–callee round-trip WhatsApp targeting implicitly minimizes.

#	Caller	Callee	Cand. 1 (selected)	Cand. 2	Cand. 3
1	australia-east	australia-east	**Sydney(20)**	Singapore(380)	Tokyo(440)
2	australia-east	central-india	**Sydney(140)**	Chennai(159)	Singapore(206)
3	australia-east	central-us	**Seattle(380)**	SanJose(400)	LosAngeles(410)
4	australia-east	chile-central	**Sydney(530)**	Santiago(530)	LosAngeles(620)
5	australia-east	east-us	**Seoul(326)**	Singapore(373)	Seoul(378)
6	australia-east	japan-east	**Tokyo(120)**	Seoul(164)	Seoul(191)
7	australia-east	malaysia-west	**Singapore(98)**	Seoul(198)	Tokyo(213)
8	australia-east	poland-central	**Seoul(361)**	Seoul(362)	Tokyo(386)
9	australia-east	south-africa-north	**Singapore(266)**	Singapore(323)	Singapore(340)
10	australia-east	south-central-us	**Singapore(346)**	Mumbai(367)	Singapore(380)
11	australia-east	uae-north	**Mumbai(180)**	Singapore(214)	Mumbai(228)
12	australia-east	uk-south	**Marseille(257)**	Milan(259)	Zurich(269)
13	australia-east	west-us	**Seoul(275)**	London(385)	Marseille(416)
14	central-india	australia-east	**Seoul(356)**	Singapore(372)	Mumbai(390)
15	central-india	central-india	**Mumbai(9)**	Amsterdam(269)	Singapore(300)
16	central-india	central-us	**Mumbai(213)**	Singapore(315)	Singapore(335)
17	central-india	chile-central	**Mumbai(319)**	Singapore(342)	Rio(370)
18	central-india	east-us	**Seoul(311)**	Seoul(324)	Singapore(345)
19	central-india	japan-east	**Seoul(118)**	SanJose(352)	Dallas(357)
20	central-india	malaysia-west	**Mumbai(57)**	Seoul(192)	Seoul(282)
21	central-india	poland-central	**Mumbai(139)**	Singapore(297)	Frankfurt(310)
22	central-india	south-africa-north	**Mumbai(216)**	Singapore(284)	Marseille(298)
23	central-india	south-central-us	**Mumbai(236)**	Amsterdam(256)	Dallas(278)
24	central-india	uae-north	**Mumbai(42)**	Singapore(188)	Seoul(261)
25	central-india	uk-south	**Mumbai(140)**	Singapore(333)	Seoul(378)
26	central-india	west-us	**Mumbai(246)**	Singapore(265)	SanJose(295)
27	central-us	australia-east	**Seoul(339)**	Frankfurt(354)	London(359)
28	central-us	central-india	**Frankfurt(265)**	Marseille(281)	Seattle(293)
29	central-us	central-us	**Chicago(60)**	Dallas(80)	Atlanta(80)
30	central-us	chile-central	**Dallas(112)**	Singapore(577)	Singapore(606)
31	central-us	east-us	**Dallas(25)**	Atlanta(40)	Chicago(55)
32	central-us	japan-east	**Chicago(148)**	Seattle(155)	SanJose(157)
33	central-us	malaysia-west	**Tokyo(215)**	Seoul(229)	Seoul(230)
34	central-us	poland-central	**Amsterdam(240)**	Frankfurt(250)	London(270)
35	central-us	south-africa-north	**Marseille(490)**	Milan(500)	London(510)
36	central-us	south-central-us	**Dallas(35)**	Chicago(45)	Atlanta(55)
37	central-us	uae-north	**Marseille(211)**	Singapore(290)	Singapore(356)
38	central-us	uk-south	**London(97)**	London(110)	Amsterdam(118)
39	central-us	west-us	**Dallas(51)**	Chicago(62)	SanJose(70)
40	chile-central	australia-east	**Atlanta(296)**	Ashburn(373)	London(500)
41	chile-central	central-india	**Santiago(403)**	Rio(425)	Mumbai(455)
42	chile-central	central-us	**LosAngeles(180)**	Santiago(180)	Dallas(205)
43	chile-central	chile-central	**BuenosAires(78)**	Rio(115)	Bogota(212)
44	chile-central	east-us	**Rio(180)**	Bogota(182)	BuenosAires(194)
45	chile-central	japan-east	**LosAngeles(530)**	Seattle(530)	SanJose(530)
46	chile-central	malaysia-west	**Singapore(620)**	LosAngeles(670)	Tokyo(720)
47	chile-central	poland-central	**Atlanta(215)**	LosAngeles(306)	Frankfurt(232)
48	chile-central	south-africa-north	**Atlanta(328)**	Ashburn(335)	Santiago(382)
49	chile-central	south-central-us	**Atlanta(126)**	Santiago(128)	Ashburn(147)
50	chile-central	uae-north	**Ashburn(279)**	Atlanta(285)	Frankfurt(297)
51	chile-central	uk-south	**Ashburn(189)**	Atlanta(194)	Frankfurt(219)
52	chile-central	west-us	**Atlanta(141)**	Ashburn(171)	Frankfurt(340)
53	east-us	australia-east	**Sydney(410)**	LosAngeles(440)	SanJose(440)
54	east-us	central-india	**London(224)**	Chicago(248)	Milan(253)
55	east-us	central-us	**Chicago(31)**	SanJose(113)	Seoul(407)
56	east-us	chile-central	**Chicago(145)**	SanJose(221)	Seattle(235)
57	east-us	east-us	**Chicago(54)**	SanJose(134)	Seattle(143)
58	east-us	japan-east	**Tokyo(162)**	Phoenix(162)	SanJose(172)
59	east-us	malaysia-west	**Tokyo(234)**	Phoenix(243)	SanJose(257)
60	east-us	poland-central	**Amsterdam(200)**	London(210)	Frankfurt(230)
61	east-us	south-africa-north	**London(200)**	Atlanta(220)	Marseille(245)
62	east-us	south-central-us	**Dallas(40)**	London(191)	Milan(236)
63	east-us	uae-north	**London(182)**	Milan(185)	FCO(203)
64	east-us	uk-south	**Chicago(122)**	Dallas(138)	Seattle(216)
65	east-us	west-us	**Chicago(151)**	Dallas(165)	Ashburn(178)
66	japan-east	australia-east	**Tokyo(101)**	Sydney(115)	Seoul(125)
67	japan-east	central-india	**Mumbai(120)**	Singapore(135)	Singapore(145)

Continued from previous page

#	Caller	Callee	Cand. 1 (selected)	Cand. 2	Cand. 3
68	japan-east	central-us	**Tokyo(146)**	Seoul(194)	Seoul(222)
69	japan-east	chile-central	**Tokyo(281)**	Seoul(300)	Seoul(331)
70	japan-east	east-us	**Tokyo(164)**	Seoul(214)	Seoul(249)
71	japan-east	japan-east	**Tokyo(20)**	Seoul(120)	Singapore(300)
72	japan-east	malaysia-west	**Tokyo(82)**	SanJose(294)	Phoenix(306)
73	japan-east	poland-central	**Singapore(238)**	Tokyo(253)	Frankfurt(252)
74	japan-east	south-africa-north	**Seoul(300)**	Dallas(378)	LosAngeles(415)
75	japan-east	south-central-us	**Seoul(205)**	Singapore(289)	Dallas(220)
76	japan-east	uae-north	**Singapore(156)**	Seoul(170)	Mumbai(170)
77	japan-east	uk-south	**Marseille(231)**	Tokyo(231)	Singapore(236)
78	japan-east	west-us	**Tokyo(110)**	Singapore(244)	Marseille(353)
79	malaysia-west	australia-east	**Seoul(215)**	Marseille(372)	London(424)
80	malaysia-west	central-india	**Singapore(115)**	Mumbai(128)	Singapore(140)
81	malaysia-west	central-us	**Singapore(250)**	Mumbai(276)	Dallas(265)
82	malaysia-west	chile-central	**Singapore(343)**	Singapore(351)	Seoul(387)
83	malaysia-west	east-us	**Singapore(233)**	Singapore(245)	Ashburn(260)
84	malaysia-west	japan-east	**Singapore(75)**	Singapore(90)	Tokyo(108)
85	malaysia-west	malaysia-west	**Singapore(40)**	Mumbai(300)	Tokyo(320)
86	malaysia-west	poland-central	**Singapore(189)**	Chennai(212)	Singapore(235)
87	malaysia-west	south-africa-north	**Singapore(183)**	Chennai(276)	Singapore(3302)
88	malaysia-west	south-central-us	**Singapore(201)**	Frankfurt(281)	Milan(286)
89	malaysia-west	uae-north	**Singapore(88)**	Singapore(147)	Seoul(213)
90	malaysia-west	uk-south	**Milan(174)**	Zurich(175)	Singapore(236)
91	malaysia-west	west-us	**Singapore(185)**	Milan(316)	Zurich(317)
92	poland-central	australia-east	**Frankfurt(324)**	Singapore(345)	Sydney(360)
93	poland-central	central-india	**Frankfurt(143)**	Amsterdam(193)	Frankfurt(158)
94	poland-central	central-us	**Amsterdam(124)**	Milan(138)	Marseille(151)
95	poland-central	chile-central	**Amsterdam(218)**	Mumbai(429)	Santiago(230)
96	poland-central	east-us	**Frankfurt(107)**	London(107)	Amsterdam(108)
97	poland-central	japan-east	**Mumbai(470)**	Tokyo(490)	Singapore(500)
98	poland-central	malaysia-west	**Milan(191)**	London(199)	Dallas(341)
99	poland-central	poland-central	**Frankfurt(80)**	Amsterdam(80)	Milan(140)
100	poland-central	south-africa-north	**Amsterdam(187)**	Frankfurt(192)	London(192)
101	poland-central	south-central-us	**Amsterdam(134)**	London(140)	Frankfurt(141)
102	poland-central	uae-north	**Milan(115)**	Atlanta(295)	Dallas(333)
103	poland-central	uk-south	**Frankfurt(36)**	Zurich(54)	Milan(56)
104	poland-central	west-us	**Frankfurt(162)**	Zurich(185)	Milan(188)
105	south-africa-north	australia-east	**Frankfurt(436)**	London(473)	Marseille(450)
106	south-africa-north	central-india	**Frankfurt(300)**	Milan(318)	Marseille(315)
107	south-africa-north	central-us	**Marseille(490)**	Frankfurt(500)	London(510)
108	south-africa-north	chile-central	**Marseille(402)**	Singapore(524)	Singapore(532)
109	south-africa-north	east-us	**Marseille(278)**	Singapore(407)	Singapore(463)
110	south-africa-north	japan-east	**Singapore(254)**	Singapore(268)	Tokyo(285)
111	south-africa-north	malaysia-west	**Singapore(189)**	Mumbai(205)	Marseille(220)
112	south-africa-north	poland-central	**Frankfurt(330)**	Amsterdam(350)	Milan(350)
113	south-africa-north	south-africa-north	**Milan(325)**	London(329)	Marseille(350)
114	south-africa-north	south-central-us	**London(281)**	Milan(295)	Marseille(312)
115	south-africa-north	uae-north	**Marseille(100)**	Mumbai(115)	Mumbai(128)
116	south-africa-north	uk-south	**Frankfurt(181)**	Zurich(187)	Milan(187)
117	south-africa-north	west-us	**London(303)**	Frankfurt(308)	Marseille(320)
118	south-central-us	australia-east	**Seoul(289)**	Seoul(291)	Singapore(338)
119	south-central-us	central-india	**Seoul(275)**	Seoul(279)	Singapore(309)
120	south-central-us	central-us	**Dallas(30)**	Atlanta(45)	Chicago(55)
121	south-central-us	chile-central	**Dallas(119)**	Atlanta(135)	Miami(148)
122	south-central-us	east-us	**Atlanta(70)**	Ashburn(80)	Dallas(90)
123	south-central-us	japan-east	**Dallas(147)**	Seattle(160)	Chicago(174)
124	south-central-us	malaysia-west	**Tokyo(206)**	Seoul(229)	Seoul(241)
125	south-central-us	poland-central	**Amsterdam(270)**	Frankfurt(290)	London(300)
126	south-central-us	south-africa-north	**Marseille(242)**	Atlanta(260)	London(280)
127	south-central-us	south-central-us	**Dallas(20)**	Atlanta(38)	Milan(279)
128	south-central-us	uae-north	**Milan(228)**	Singapore(280)	Singapore(314)
129	south-central-us	uk-south	**Dallas(118)**	Seoul(364)	Seoul(397)
130	south-central-us	west-us	**LosAngeles(90)**	SanJose(90)	Dallas(100)
131	uae-north	australia-east	**Mumbai(175)**	Singapore(190)	Mumbai(205)
132	uae-north	central-india	**Mumbai(35)**	Mumbai(50)	Chennai(62)
133	uae-north	central-us	**Amsterdam(420)**	Mumbai(430)	Frankfurt(440)
134	uae-north	chile-central	**Frankfurt(299)**	FCO(312)	Mumbai(334)
135	uae-north	east-us	**Frankfurt(180)**	Marseille(180)	Milan(185)
136	uae-north	japan-east	**Mumbai(310)**	Singapore(380)	Tokyo(430)
137	uae-north	malaysia-west	**Singapore(93)**	Tokyo(226)	Milan(242)
138	uae-north	poland-central	**Marseille(134)**	Singapore(250)	Tokyo(409)
139	uae-north	south-africa-north	**Marseille(100)**	Mumbai(118)	Mumbai(130)
140	uae-north	south-central-us	**Milan(218)**	Frankfurt(218)	Zurich(222)
141	uae-north	uae-north	**Milan(167)**	Zurich(173)	Frankfurt(185)

Continued from previous page

#	Caller	Callee	Cand. 1 (selected)	Cand. 2	Cand. 3
142	uae-north	uk-south	**London(104)**	Frankfurt(107)	Zurich(108)
143	uae-north	west-us	**London(231)**	Frankfurt(232)	Milan(234)
144	uk-south	australia-east	**London(250)**	Frankfurt(320)	Amsterdam(322)
145	uk-south	central-india	**Amsterdam(143)**	Mumbai(158)	London(172)
146	uk-south	central-us	**London(99)**	Marseille(126)	Seoul(428)
147	uk-south	chile-central	**London(430)**	Ashburn(450)	Amsterdam(480)
148	uk-south	east-us	**London(82)**	Frankfurt(103)	Seoul(459)
149	uk-south	japan-east	**London(229)**	FCO(254)	Milan(261)
150	uk-south	malaysia-west	**Marseille(169)**	Singapore(175)	Seoul(313)
151	uk-south	poland-central	**London(28)**	Marseille(58)	Seoul(512)
152	uk-south	south-africa-north	**London(166)**	Marseille(193)	Seoul(561)
153	uk-south	south-central-us	**Amsterdam(122)**	London(135)	Dallas(150)
154	uk-south	uae-north	**London(109)**	Frankfurt(110)	Mumbai(122)
155	uk-south	uk-south	**London(4)**	Frankfurt(33)	Zurich(43)
156	uk-south	west-us	**London(139)**	Amsterdam(148)	Frankfurt(157)
157	west-us	australia-east	**Sydney(310)**	SanJose(320)	LosAngeles(330)
158	west-us	central-india	**Frankfurt(272)**	Milan(311)	Amsterdam(322)
159	west-us	central-us	**SanJose(100)**	Seattle(110)	LosAngeles(120)
160	west-us	chile-central	**Dallas(561)**	Santiago(580)	Miami(605)
161	west-us	east-us	**Ashburn(45)**	Chicago(58)	Dallas(70)
162	west-us	japan-east	**Seattle(220)**	SanJose(230)	Tokyo(230)
163	west-us	malaysia-west	**Singapore(182)**	Tokyo(198)	Singapore(210)
164	west-us	poland-central	**Frankfurt(160)**	Marseille(184)	Milan(186)
165	west-us	south-africa-north	**Frankfurt(306)**	Milan(319)	Marseille(322)
166	west-us	south-central-us	**LosAngeles(90)**	SanJose(90)	Phoenix(90)
167	west-us	uae-north	**Frankfurt(233)**	Milan(240)	Mumbai(248)
168	west-us	uk-south	**Dallas(133)**	Chicago(142)	Milan(171)
169	west-us	west-us	**SanJose(20)**	LosAngeles(40)	Seattle(60)